Lecture 21 Practice Questions and Solutions

Question 1 (q1_sol.py)

Sold Out!

Orders for the campus coffee shop are kept in a list of strings, oldest order first. For example:

```
orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
```

Sometimes the shop runs out of an item, and so those items need to be removed from the order list. The barista also needs to know how many orders were removed.

Write a function called remove_sold_out(orders, bad_item) that:

- removes *every* copy of bad_item from the list orders, using the list's remove method
- returns (not prints!) the number of items removed

The function should change the orders list itself in-place, i.e. it should NOT make a new list.

Your remove_sold_out(orders, bad_item) function should work with code like this:

```
orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
n = remove_sold_out(orders, 'oat latte')
print(n)        # prints 3
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']

# orders is now: ['latte', 'muffin', 'tea', 'scone']
n = remove_sold_out(orders, 'hot chocolate')
print(n)        # prints 0
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

Sample Solution

```
def remove_sold_out(orders, bad_item):
    count = 0
    while bad_item in orders:
        orders.remove(bad_item)
        count += 1
    return count

orders = ['latte', 'muffin', 'oat latte', 'tea', 'oat latte',
          'scone', 'oat latte']
n = remove_sold_out(orders, 'oat latte')
print(n)        # prints 3
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

```
# orders is now: ['latte', 'muffin', 'tea', 'scone']
n = remove_sold_out(orders, 'hot chocolate')
print(n)        # prints 0
print(orders)   # prints ['latte', 'muffin', 'tea', 'scone']
```

Question 2 (q2_sol.py)

SFU gives semesters a code number as follows:

```
Semester   Code
---------------
 Spring      1
 Summer      4
 Fall        7
```

Write a program that stores the semesters and their codes in a dictionary. The semesters are the keys, and the codes are the values.

Then write a program that repeatedly asks the user to enter a semester, and, using the dictionary, prints the corresponding code. It should stop when they press the <return> key. If the user enters an unknown semester, then print a helpful error message.

Important: the *case* of the semester name does not matter. So, for example, "spring", "Spring", and "SPRING" should all be treated the same. The output the semester name is always capitalized, e.g. "Summer", "Spring, "Fall".

Sample Run

```
Enter semester: Spring
Spring code: 1

Enter semester: SUMMER
Summer code: 4

Enter semester: Winter
Unknown semester

Enter semester: <return>
All done!
```

Sample Solution

```
codes = {'spring': 1, 'summer': 4, 'fall': 7}

while True:
    semester = input('Enter semester: ')
    semester = semester.strip().lower()
    if semester == '':
        break
    elif semester in codes:
        code = codes[semester]
        print(f'{semester.capitalize()} code: {code}')
    else:
        print('Unknown semester')
```

```
    print()

print('All done!')
```